Appendices

A. Unicode, UTF-8 and BPE

Both Wikipedia page and [LB24] have a helpful discussion of UTF-8. Here we do a small refresher for the sake of completeness.

Unicode is basically a list of all characters that were agreed to be important and useful enough to have their own label (for an example that was not, see CE marking). The label (also called a code point) is the index of the character in this list, and is usually written in hexadecimal form as `U+abcdef` or `U+cdef` if both `ab` are zeroes, e.g. `U+004B` is a Latin capital letter `K` and `U+041A` is a Cyrillic capital letter `К`.

UTF-8 is an encoding that allows to translate the code points into machine-readable format, bytes. It would be very inefficient [Sha48] to store and transfer all the code points as $4 * 6/8 = 3$ bytes (some symbols are used more frequently and others less frequently), so it was decided to split all code points into 4 such groups that an element of each group gets encoded by 1, 2, 3 or 4 bytes. For example, since both Unicode and UTF-8 were designed by an English-centric community, it was decided that 1-byte range should contain ASCII.

In general, the conversion between Unicode code point `U+uvwxyz` (hex) and UTF-8 (binary) takes the following form:

- `U+0000:U+007F` → `0yyyzzzz`
- `U+0080:U+07FF` → `110xxxyy 10yyzzzz`
- `U+0800:U+FFFF` → `1110wwww 10xxxxyy 10yyzzzz`
- `U+010000:U+10FFFF` → `11110uvv 10vvwwww 10xxxxyy 10yyzzzz`

Stated as above it is not really a bijection: for example, there are some duplicate cases when several sequences of UTF-8 bytes would correspond to the same Unicode code points. For example, `1100000y 10yyzzzz` would correspond to the same code point as `0yyyzzzz`. Such bytes as `1100000y` (`C0` and `C1`) and some other never appear in Unicode. Therefore, in our approach to constructing encodings we need to be wary and do some cleaning which we achieve by simply trying to decode the UTF-8 bytes to string and dropping the encoding in case of an error.

All above is relevant since the tokenizers in LLMs are often trained directly on bytes obtained from encoding text as UTF-8. We can't really think of a clearer reference than [Kar24].

B. Evaluation details

We mostly use external providers in our evaluation. To help with reproducibility we list exact settings (model provider, model name, max tokens, size of $\mathcal{C}$ and any extra parameters) we used in our evaluation of understanding in Table 3. Note that for some models we didn't test the whole set of $\mathcal{C}$: it was either due to prohibitively high cost (o1-mini) or rate limits (gpt-4o, llama-3.3).

Table 3: Summary of understanding evaluation details.

	Evaluation parameters				
Model	**provider**	**model name**	**max tokens**	$\lvert\mathcal{C}\rvert * \lvert I_L\rvert$	**parameters**
Claude-3.5 Haiku	Anthropic	claude-3-5-haiku-20241022	200	8684	
Claude-3.5 Sonnet (New)	Anthropic	claude-3-5-sonnet-20241022	200	8684	
Claude-3.5 Sonnet Old	Anthropic	claude-3-5-sonnet-20240620	200	8684	
Claude-3.7 Sonnet	Anthropic	claude-3-7-sonnet-20250219	200	8684	thinking disabled
gpt-4o mini	OpenAI	gpt-4o-mini-2024-07-18	200	8684	seed = 0
gpt-4o	OpenAI	gpt-4o-2024-08-06	300	2832	seed = 2025
o1-mini	OpenAI	o1-mini-2024-09-12	3000	1880	seed = 2025
Llama-3.3 70B	Together	meta-llama/Llama-3.3-70B-Instruct-Turbo-Free	1000	2890	seed = 2025
GigaChat-Max	GigaChat	GigaChat-Max:1.0.26.20	100	1532	seed = 2025
Vikhr-Llama-3.2 1B	HuggingFace	Vikhrmodels/Vikhr-Llama-3.2-1B-Instruct	150	8684	
Qwen2.5 32B	Nebius	Qwen/Qwen2.5-32B-Instruct-fast	200	8684	
Qwen2.5 1.5B	Nebius	Qwen/Qwen2.5-1.5B-Instruct	200	8684	
Phi-3.5-mini	Nebius	microsoft/Phi-3.5-mini-instruct	200	8684	

Similar details about our evaluation of attacks are provided in Table 4.

Table 4: Summary of attack evaluation details.

	Evaluation parameters				
Model	**provider**	**model name**	**max tokens**	$\lvert\mathcal{H}\rvert$	**parameters**
Claude-3.5 Sonnet (New)	Anthropic	claude-3-5-sonnet-20241022	1000	100	$\sim$ break@50
gpt-4o	OpenAI	gpt-4o-2024-08-06	1000	30	break@10
gpt-4o mini	OpenAI	gpt-4o-mini-2024-07-18	1000	30	break@3

All raw logs are available at this URL.

C. Additional details

Understanding

Table 5: Understanding rate for various **encoded** sequences using rule-based heuristic.

encoding type	**2b**		**3b**		**4b**	
nudge	**-**	+	**-**	+	**-**	+
Model	**Understanding rate of various $\mathcal{C}$**					
Claude-3.5 Haiku	0.000	0.000	0.013	0.067	0.021	0.106
Claude-3.5 Sonnet (New)	0.000	0.000	0.021	0.075	0.021	0.031
Claude-3.5 Sonnet (Old)	0.000	0.000	0.017	0.138	0.042	0.060
Claude-3.7 Sonnet	0.000	0.000	0.083	0.225	0.069	0.304
DeepSeek-R1-DLlama-70B	0.000	0.000	0.008	0.008	0.000	0.001
GigaChat-Max	0.000	0.083	0.004	0.006	0.000	0.000
Llama-3.3 70B	0.000	0.000	0.008	0.013	0.006	0.022
Phi-3.5 mini	0.000	0.000	0.000	0.000	0.000	0.000
Qwen2.5 1.5B	0.000	0.000	0.000	0.000	0.000	0.000
Qwen2.5 32B	0.000	0.000	0.008	0.008	0.005	0.010
Vikhr-Llama-3.2 1B	0.000	0.000	0.000	0.000	0.000	0.000
gpt-4o	0.000	0.000	0.013	0.013	0.009	0.009
gpt-4o mini	0.000	0.000	0.008	0.008	0.000	0.003
o1-mini	0.000	0.000	0.013	0.046	0.003	0.007

Table 6: Understanding count for various **encoded** sequences using rule-based heuristic.

	Understanding count of various $\mathcal{C}$ by nudge			
Model	**understood**		**size**	
Model	**no nudge**	**nudge**	**no nudge**	**nudge**
Claude-3.5 Haiku	90	449	4342	4342
Claude-3.5 Sonnet (New)	93	144	4342	4342
Claude-3.5 Sonnet (Old)	177	277	4342	4342
Claude-3.7 Sonnet	301	1299	4342	4342
DeepSeek-R1-DLlama-70B	2	3	1246	1246
GigaChat-Max	2	4	766	766
Llama-3.3 70B	9	29	1445	1445
Vikhr-Llama-3.2 1B	9	29	4342	4342
Phi-3.5 mini	0	0	4342	4342
Qwen2.5 1.5B	0	0	4342	4342
Qwen2.5 32B	24	41	4342	4342
gpt-4o	14	13	1416	1416
gpt-4o mini	3	15	4342	4342
o1-mini	5	16	990	990